Semana 1 - Del computador al microcontrolador

Objetivos: Explicar los bloques de un microcontrolador (CPU, buses, memorias y periféricos) y diferenciarlos de un microprocesador.

Binario y hexadecimal (es un lenguaje nativo, en páginas web).
↓ ↓
Computador mezcla de humano y PC

octale -> linux
CPU -> microprocesador y microcontrolador

Diferencias:
- ninguna, solo la definición
- microprocesador -> lo tienen las computadoras, celulares.
- microcontrolador -> lo tienen la lavadora, sensor, alarma

Memoria
Es volatil y no volatil
Sirve para almacenar.
- RAM, ROM, Flash, Eeprom, SRAM, SDRAM, DRAM, caché, flip-flop, capacitor
silicio -> en los semiconductores

Buses
- Van y vienen
- Transportan
- 3 tipos de buses -> Control
-> Datos
-> Dirección

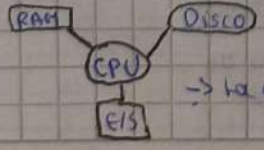
- Ancho de bus
- Bidireccional y unidireccional.

Ensamblador -> lenguaje de bajo nivel

¿Qué es un sistema embebido?
Un aparato con un microcontrolador dentro, dedicado a una tarea fija
Hace una sola cosa, siempre lo mismo, poco memoria, poco consumo, funciona sin que el usuario lo note, ejemplo la lavadora, control remoto, router, termostato, automovil y alarma, en el auto la radio.

Microprocesador frente a microcontrolador.

Microprocesador -> es el cerebro

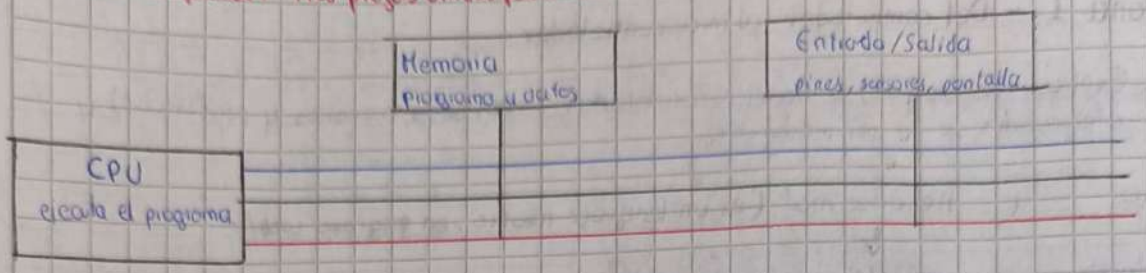


-> la CPU sola

Microcontrolador -> tiene todo dentro pero cumple una función pequeña.

CPU Flash
SRAM Periféricos. -> Todo en un chip.

Todo computador: tres piezas unidos por buses



- Bus de direcciones: ¿qué casillero?
- Bus de datos: el contenido
- Bus de control: ¿leer o escribir?

Cuanto cabe KB y MB

1 byte	1KB	1MB
8 bits	1024 bytes	1024 KB

Dentro de la CPU

- ALU → suma, resta, AND, XOR, OR, suma de bits.
- Registros → casilleros internos muy rápidos; El AUR tiene 32: x0 a x31, son muy rápidos
- Unidad de control → lee cada instrucción y decide que hacer.
- Contador de programa → Guarda la dirección de la siguiente instrucción.
- Núcleo → es una CPU completa, ejm, la CPU se encarga de una función.

La CPU repite tres pasos al ritmo de reloj:

Buscar → Decodificar → Ejecutar

El reloj



16 MHz → 16 mill de hz
16 mill de pulsos por segundo, cada pulso dura 62.5 ns

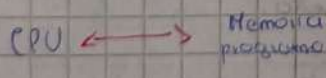
Buscar: traer la instrucción desde la memoria
Decodificar: entender que pide
Ejecutar: hacerlo

Tres buses tres preguntas

- Bus de direcciones → casillero
  datos → El contenido
  control → leer o escribir.

Von Neumann frente a Harvard.

Von Neumann



Harvard



Tres memorias: ¿qué pasa al apagar?

Flash -> No se borra, guarda el programa, se reescribe al subir un programa desde el IDE, un libro.
SRAM -> Se borra, guarda variables mientras el programa corre. Rápida y pequeña. Como una pizarra
EEPROM -> No se borra, guarda pocos datos que el programa cambia y deben sobrevivir al apagado. Una libreta.

Escribir desgasta: Flash y EEPROM tienen límite

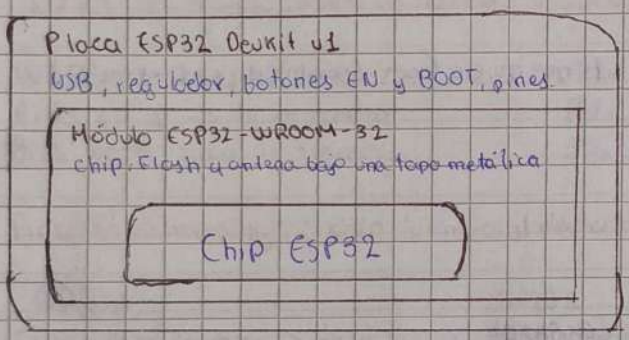
10000 ciclos garantizados de la Flash ✗ Un ciclo: borrar y volver a escribir la misma celda. Leer no desgasta.

100000 ciclos garantizados de la EEPROM

No se escribe dentro de loop.

ESP32: 32 bits, 2 núcleos, hasta 240 MHz, 3,3V (no tolera 5V), 4MB Flash, 320 KB SRAM, WiFi, Bluetooth
ATmega328P: 8 bits, 1 núcleo, 16MHz, 5V, 32KB Flash, 2KB SRAM, 1KB EEPROM, sin WiFi

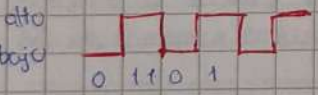
Chip. Módulo y Placa.



Arduino no es un chip.
Es una plataforma.

Bits, operaciones bit a bit y E/S mapeada a memoria

Un bit es un nivel de voltaje



Un byte: 8 bits, cada uno con su peso

128	64	32	16	8	4	2	1	
0	0	1	0	0	0	0	0	= 32
0	0	0	0	0	1	0	1	= 4 + 1 = 5

Hexadecimal: 4 bits en un solo símbolo

0 0 1 0 0 0 0 0 0010 0000 = 0x20
↓ ↓
2 0

Compuertas lógicas, un bit a la vez.

A	B	AND &	OR |	XOR ^	NOT A ~
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

AND -> 1 solo si ambos son 1
OR -> 1 si alguno es 1
XOR -> 1 si son distintos
NOT -> invierte

Bit a bit: columna por columna

```
     1 1 0 0 1 0 1 0
AND  0 0 0 0 1 1 1 1
 =   0 0 0 0 1 1 1 1
```

La CPU hace 8 columnas a la vez, en una sola instrucción.

La máscara: 1 << n señala un bit

```
0 0 0 0 0 0 0 1
        <------  mover 5 lugares a la izquierda
0 0 1 0 0 0 0 0
```

Una máscara es un byte con 1 solo en los bits que quiero tocar. Escribir 1 << 5 es más legible que 0x20: dice "el bit 5".

Demostración de C a bits.

Lenguaje C — PORTB |= (1 << 5)
Ensamblador — sbi 0x05, 5
Código máquina en la Flash — 1001 1010 0010 1101 (0x9A2D)

RISC y CISC; 8 y 32 bits.

RISC -> Pocas instrucciones, simples; casi todas en un ciclo de reloj.
CISC -> Muchas instrucciones, algunas muy elaboradas

Tres registros por puerto

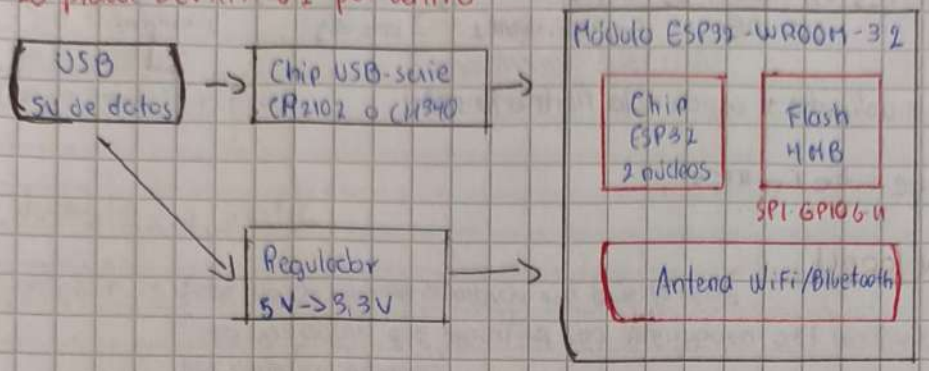
Registro	Para qué sirve.	Bit en 1 significa
DDRB	Dirección: entrada o salida	Salida
PORTB	Salida: valor que sale. Entrada: pull-up	5V en el pin, o pull-up activo
PINB	Lee el voltaje real del pin	El pin está en alto.

Los cuatro estados de un pin.

DDRxn	PORTxn	Estado del pin	Ejemplo
0	0	Entrada flotante: lee al azar	Nada conectado
0	1	Entrada con pull-up	Botón a GND
1	0	Salida en 0 (0V)	LED apagado
1	1	Salida en 1 (5V)	LED encendido

Semana 2 - Parte 1 - Del AVR al ESP32

La placa DevKit V1 por dentro



Multiplexor -> circuito que recibe muchas entradas

GPIO -> Entrada y salida de proposito general

W1TS y W1TC: notas de "encender" y "apagar"

W1TS - write 1 to set
Bits en 1 -> se encienden
Bits en 0 -> no se tocan

W1TC - write 1 to clear.
Bits en 1 -> se apagan
Bits en 0 -> no se tocan

Pines de Strapping: el chip los mira al despertar.

GPIO0
En bajo al arrancar -> modo de carga
Es el botón BOOT

GPIO2
Debe estar en bajo o al aire para cargar
Suele tener el led azul de la placa

GPIO12
En Alto al arrancar -> flash a 1,8 V
La placa no arranca

GPIO5 y GPIO15 también se leen al arrancar
No se conectan entradas a pines de strapping

3,3V: los límites eléctricos

$R = (3,3V - Vf) / I$
Led Rojo, 330 Ω -> (3,3 - 2,0)/330 ≈ 4 mA

≤ 10 mA
diseño de un pin

3,3 V
máximo en una entrada

0,3 V
lo que queda a un led azul

Del reset a loop()

Bootloader ROM → 2.º bootloader (Flash) → app_main() ESP-IDF → loopTask FreeRTOS, núcleo 1 → setup() una vez → loop() siempre ↑

Antes de setup(): los globales inicializados se copian de la Flash a la SRAM.

Todo sketch del ESP32 ya corre sobre FreeRTOS.

El superlazo: setup() y loop()

setup() · una vez

↓

Leer entradas → Decidir

↑ ↓

Escribir salidas

loop() → para siempre

Lo que cuesta un loop vacío

240 MHz

girando aunque loop() no tenga nada que hacer.

Activo

La CPU ejecuta instrucciones sin parar: decenas de mA

Deep sleep.

Casi todo se apaga y un temporizador lo despierta: del orden de µA

Tipos de datos

Una variable es una caja: tamaño e interpretación

1 1 0 0 1 1 0 0

↙ ↘

uint8_t
Sin signo
= 204

int8_t
Con signo (bit alto = signo)
= -52

stdint.h: el tamaño va en el nombre

int8_t = -128 ... 127

int16_t = -32768 ... 32767

int32_t = ≈ ±2.1 × 10⁹

uint8_t
0 ... 255

uint16_t = 0 ... 65535

uint32_t = 0 ... ≈ 4.29 × 10⁹

Desbordamiento

253 -> 254 -> 255 -> 0 -> 1

int = millis()
Uno: falla a los ~32s
ESP32: falla a los ~24.8 días
Usar uint32_t

Sin signo
Da la vuelta.
Comportamiento definido

Con signo
Comportamiento indefinido
No confiar en él.

Volatile y static

Volatile -> Este valor puede cambiar sin que lo veas.
Para variables que modifica una interrupción o el hardware
No significa atómico

Static -> Dentro de una función: conserva su valor entre llamadas.
En un archivo: privada de ese archivo.

delay(): el cocinero que mira el horno

loop con delay(500)

lee botón | delay(500) bloqueado | lee botón | delay(500) bloqueado

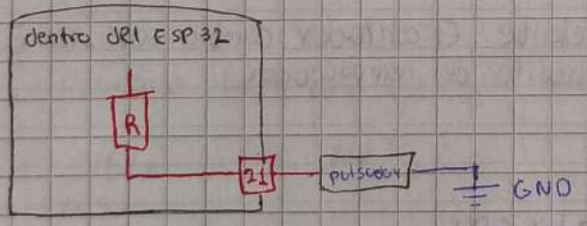
↑
presiono y suelta -> nadie leyó: pulsación perdida

millis(): mirar el reloj de pared

- Leer el reloj
- Resta ¿cuánto pasó?
- Si ya toca: actuar y anotar
- Seguir con lo demás

Un temporizador de hardware cuenta siempre; millis() solo mira los tiempos (semana 3)

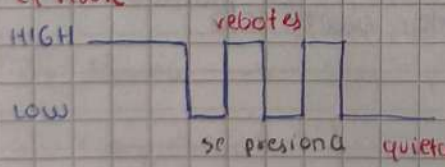
Pull-up: darle un pin a un nivel definido



Pulsador | Lectura
suelto | HIGH (1)
presionado | LOW (0)

GPIO 34-39 sin pull-up interna.

El rebote



se presiona quieto 30 ms -> válido

Antirebote -> aceptar el cambio solo si la señal se queda quieta entre 20 y 50 ms.

Semana 8 - Temporizadores de hardware en el ESP32

El número de GPIO es un nombre no una posición

30 pines en la placa devkit v1

Los GPIO 34 a 39 solo sirven como entrada

0 - 255 -> 300 - 256 = 44

Temporizador -> Es un circuito interno de microcontrolador que cuenta pulsos de reloj.
↓
Es un periférico

60 MHz -> 60 veces en un segundo

$f = \frac{1}{T_s}$ $T = \frac{1}{f}$ $\frac{1}{60} = 0,016s$

Cada pulso vale 0,016s.

delay() = La CPU se queda esperando sin hacer nada

millis() = Todos siguen trabajando, la CPU pregunta "¿ya pasó?" en cada vuelta de loop().

Temporizador -> Un circuito aparte y avisa a la hora exacta.

El contador es un registro del temporizador.

Registro: Espacio pequeño de almacenamiento de bits. Los periféricos tienen los suyos.

Contador: Registro del temporizador con un circuito sumador pegado.

Diferencia con la RAM: Solo guarda lo que escribe. El contador cambia solo, no está en la RAM de las variables sino dentro de circuito del temporizador.

¿Como cuenta? -> En binario

¿Quien lo arranca? ¿Quien lo detiene? -> La CPU

La CPU configura, la CPU lo habilita, el temporizador cuenta solo.

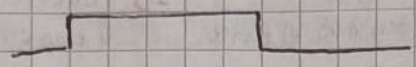
El reloj, el prescaler y las formulas:

- El reloj del temporizador: 80 millones de pulsos por segundo
  Señal que sube y baja

APB = Bus de periféricos del ESP32. Su reloj de 80 MHz, es el que llega a los tiempos se usa para comunicar los periféricos con trabaja a 80 MHz

80 MHz → 80000000 pulsos por segundo, cada pulso dura 12.5 ns

- El prescaler → un divisor antes del contador



Divisor que deja pasar al contador solo 1 de cada N pulsos de reloj. El contador avanza N veces más lento

Tick del timer: cada pulso sale del prescaler, es cuando el contador suma 1.

- Resolución → es la unidad más pequeña → duración de cada tick

El tiempo de cada pulso

resolución = 1/f f = ticks por segundo → cantidad de pulsos.

Se usa para calcular el tiempo: la escalera

altura de un peldaño = resolución
número de peldaños = alarma suena o cuando llega.

Alarma = número que se guarda en un registro cuando el contador alcanza aviso.

tiempo = alarma x resolución

despej. alarma = tiempo x frecuencia.

Ejemplo 1:

f = 1000000 ticks por segundo tiempo deseado = 0,5 s

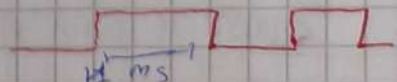
f = 1 MHz T = 1/f
t = 0,5 s

① Resolución → 1/1000000 = 1 µs

② Alarma → 0,5 x 1000000 = 500000

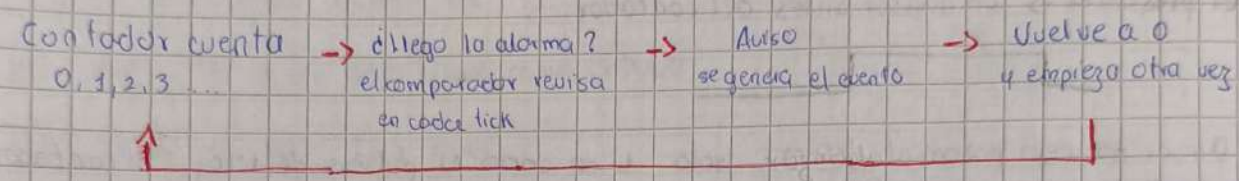
③ Comprobación → 50000 x 1 µs = 0,5 s

④ Rescaler →



Desbordamiento
No es lo mismo que la autorrecarga.

Autorrecarga. el aviso se repite solo



El ESP32 tiene 4 temporizadores de proposito general.

API -> escribe los registros por nosotros

Core -> (núcleo) -> paquete de software que se instala en el arduino IDE para programar una placa concreta. Trae el compilador y la API de esa placa.

Interrupción -> el timbre que te hace dejar lo que haces.

loop() -> ejecutando la linea 12 -> suena la alarma.

¿Qué es la ISR? Rutina de servicio de interrupción.

Callback

Dejar tu número para que te llamen

La función llama.

IRAM_ATTR -> guarda la ISR donde siempre este a la mano

Volatile. Leer siempre el valor real, no una copia.

Patron bandera: la ISR avisa, loop() trabaja

Semana 4 - Interrupciones externas en el ESP32

Conceptos ->

Sondeo (Polling) -> Preguntar una y otra vez, en el loop() si algo ya pasó.

Interrupción -> Avisa del hardware que pausa el programa para atender algo urgente

Interrupción externa -> Llega desde fuera del chip, por un pin.

ISR -> Interrupt service Routine: la función que se ejecuta al llegar el aviso

Flanco -> El instante que una señal cambia de 0 a 1 y 1 a 0

Rebote -> Varios cambios falsos y rapidísimos que produce el botón mecánico

Compilador -> Traduce C a instrucciones del chip.

Registro de la CPU -> Casillas muy rápidas dentro de la CPU donde trabaja con copia de datos.

Flash -> Memoria grande y lenta, fuera del núcleo, donde se guarda el programa.

SRAM / IRAM -> Memoria rápida dentro del chip. IRAM es la parte para instrucciones.

Caché -> Copia pequeña y rápida de lo que lo CPU leyó de la flash.

Watchdog -> "Perro guardián": reinicia el chip si algo se quedó trabado mucho tiempo

Sondeo -> En el ESP32 el loop() lee el pin en cada vuelta.

Interrupción -> Instalar un timbre, cuando suena se sale a ver y seguimos con lo que estábamos.

El ISR debe ser corto

El GPIO cambia de 1 a 0 -> Periférico GPIO detecta el cambio -> CPU pausa el loop y corre la ISR

Diferencias contra semana 3

Nivel frente a flanco

El flanco es cuando sube o cambia de estados, el nivel todo el tiempo

Tres modos -> RISING, FALLING y CHANGE

FALLING -> Ocurre al presionar, o flanco de bajada.

RISING -> Ocurre al soltar, o flanco de subida

CHANGE -> los dos estados, ISR corre dos veces por toque.

¿Que pines del ESP32 admiten interrupciones?

Pines	¿Sirven	Cuidado
16, 17, 18, 19, 21, 22, 23, 25, 26, 27, 32, 33	Sí	los recomendados
34, 35, 36, 39 (solo entrada)	Si con cuidado	No tienen pull-up interna
0, 2, 5, 12, 15 (strapping)	Evitar	Si estan presionados al encender, entra modo
6 a 11 (flash)	No	los usa la memoria del programa.

Attach Interrupt(): Función que llama a ISR ejs: button

y

no poner delay.

¿Que es una ISR?

Función normal
la llamamos en el codigo en el punto que decides

ISR
Tu nunca la llamas, la llama el hardware

Regla 1: La ISR debe ser corta.

✓ Si va en la ISR

- Subir una bandera
- Guardar el tiempo: millis()
- Sumar 1 a un contador
- digitalWrite (es rapido).

X No va en la ISR

- Serial.print / println
- delay()
- Cálculos largos
- Esperar a que algo pase

Si la ISR tarda demasiado: el watchdog

ISR larga (Serial, delay) -> Watchdog detecta que algo está trabado -> Reinicia el ESP32 arranca de nuevo -> Monitor serie "Guru Meditation Error".

El programa vive en la flash.

IRAM y DRAM: dos zonas de la misma RAM

IRAM -> Instruction RAM
Guarda instrucciones: código que la CPU ejecuta
Aquí ponemos la ISR

DRAM -> Data RAM
Guarda datos: por ejemplo, las variables globales
Aquí vive la variable alerta.

La caché

CPU ← Caché ← Flash

CPU: pide la siguiente instrucción
Caché: copia de lo leído hace poco
Flash: el programa completo, lento

El problema -> a veces la caché se apaga.

IRAM_ATTR: "guarda esta función en la IRAM"

El compilador es ahorrativo, no sabe que existe la ISR

Lo que hace volatile

```
volatile bool alerta = false;
```

"Esta variable puede cambiar a tus espaldas. No confíes en tu copia: ve a la RAM cada vez que la uses".